claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-a501f4bf5d2bcdfd2.jsonl`
- SHA-256: `6778fd2c2e49bdbec9f27c9436e0e2d0fd411ebd8a753b1c0a0e0f9e6c8630a5`
- Source modified: `2026-06-22T15:46:27+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

Transcript

1. user (2026-06-22T15:43:39.279Z)

You are doing an adversarial code review of a small, surgical change in the git worktree at `C:\Users\avidu\Projects\bhi-picker` (a FastAPI badminton rally-detection engine). Find REAL bugs only ? correctness, security, default-OFF/backward-compat regressions, or logic errors. Do NOT suggest style nits or speculative refactors. Be skeptical and try to break it.

The change adds a per-request "compute picker" to the `/api/process` flow so a request can force in-process ("local") or Cloud Run dispatch ("cloud"), plus a `cloud_available` capability flag. Read these to ground yourself:

1. `git -C C:\Users\avidu\Projects\bhi-picker diff` ? the full diff (3 files: backend/compute/__init__.py, backend/main.py, tests/test_api_cloud_dispatch.py).
2. In `backend/main.py`: the functions `_cloud_available()`, `_maybe_dispatch_remote()`, `capabilities()`, the `ProcessRequest` model, and how `_maybe_dispatch_remote` is called inside `_execute_processing` (look for `remote = _maybe_dispatch_remote(`). Understand the `_failed()` closure, the `play_wm`/`cand_wm` destroy-after-confirm watermark logic, and `_finalize_reingest`.
3. In `backend/compute/__init__.py`: `get_compute_backend()` with the new `target_override` param.

Specifically probe these risk areas and report your verdict on each:
- **Default-OFF invariant**: with `compute=None` (the field absent ? every existing CLI/old-SPA caller), is the behavior byte-identical to before this change? Trace it.
- **`compute="cloud"` on a server whose default IS already cloud_run but with no env vars set** ? does `_cloud_available()` correctly return True (via the compute_target short-circuit) and not falsely block?
- **`compute="local"` early-return** ? does returning None correctly fall through to the in-process segmenter, and could it skip any necessary cleanup/watermark handling that the cloud path would have done?
- **The `_failed()` call in the unconfigured-cloud branch** ? it runs `db_instance.prune_segments`; is that safe given no rows were written yet? Does it leave the DB/video-status in a coherent state?
- **`target_override` in get_compute_backend** ? `None` falls back to config; `""` (empty string) would be treated as an explicit override of "" ? returns None. Is there any caller path that could pass `""` unintentionally?
- **Capability/security**: does `_cloud_available()` or the capabilities endpoint leak any secret values (vs. mere booleans)?
- **Ordering**: in `_maybe_dispatch_remote`, the picker logic runs before the input-resolution/bucket checks. Is the precedence of failure messages sensible (config-not-available before input-type errors)?

Return a concise findings list. For each finding: severity (blocker/major/minor), the exact file:line, why it's a real bug, and a concrete fix. If you find nothing real, say so explicitly ? do not invent issues to look thorough.

2. assistant (2026-06-22T15:43:41.854Z)

I'll start by pulling the latest state and reading the diff, then ground myself in the surrounding code.

3. user (2026-06-22T15:43:44.671Z)

```
You are not currently on a branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.

    git pull <remote> <branch>

---DIFF---
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 96c5dbc..8ef0cb7 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -35,6 +35,7 @@ def get_compute_backend(
     run_client: Optional[Any] = None,
     policy: ExecutionPolicy = DEFAULT_POLICY,
     token: Optional[str] = None,
+    target_override: Optional[str] = None,
 ) -> Optional[ComputeBackend]:
     """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to run
     in-process (the default-OFF path).
@@ -43,8 +44,16 @@ def get_compute_backend(
     Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` / ``CLOUD_RUN_REGION``
     / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is injectable (tests
     pass a fake; production defaults to the real ``google-cloud-run`` client, owner-verified on M7).
+
+    ``target_override`` lets a single request force a target (the SPA compute-picker maps a
+    per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating the
+    server's configured default. ``None`` falls back to ``config.deployment.compute_target``.
     """
-    target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
+    target = (
+        target_override
+        if target_override is not None
+        else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
+    )
     if target != "cloud_run":
         return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

diff --git a/backend/main.py b/backend/main.py
index eb3f7bf..ab6a11c 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -267,6 +267,10 @@ class ProcessRequest(BaseModel):
     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for PMF
     # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older clients).
     locale: Optional[str] = None
+    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured server;
+    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ? server
+    # default (deployment.compute_target). Only honoured for these two values; see _maybe_dispatch_remote.
+    compute: Optional[str] = None

 class QueryRequest(BaseModel):
     video_id: str
@@ -370,6 +374,9 @@ def capabilities():
         "deployment": {
             "profile": getattr(deployment, "profile", "") or "",
             "compute_target": getattr(deployment, "compute_target", "") or "",
```

```
+            # item 3: can this server satisfy a per-request compute="cloud" dispatch? The SPA
shows
+            # the "run in the cloud" toggle only when true (else it's local-only).
+            "cloud_available": _cloud_available(),
         },
     }

@@ -479,6 +486,23 @@ def _ingest_remote_segments(video_id: str, outputs_uri: str,
storage_config: dic
     return n


+def _cloud_available() -> bool:
+    """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item 3).
+
+    True when the control plane is wired for Cloud Run: either the configured default target is
+    already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env coordinates
+    (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
+    are present so a forced per-request override can actually dispatch + collect a result.
+
+    Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it would
work."""
+    deployment = getattr(global_config, "deployment", None)
+    if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
+        return True
+    has_job = bool(os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"))
+    out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
+    return bool(has_job and out_root)
+
+
 def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
                            play_wm: int, cand_wm: int, notify) -> Optional[Tuple[Dict[str,
Any], bool]]:
     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
@@ -492,12 +516,6 @@ def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name:
str, va
     the in-process path stays byte-identical)."""
     from backend.compute import ComputeJobSpec, get_compute_backend

-    compute = get_compute_backend(global_config)
-    if compute is None:
-        return None  # default-OFF ? run the in-process segmenter below (unchanged)
-
-    storage_cfg = getattr(global_config, "storage", None)
-
     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
play_wm);
         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
@@ -505,6 +523,29 @@ def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name:
str, va
         return ({"video_id": video_id, "status": "failed", "message": msg,
                  "results": [r.model_dump() for r in val_results], "play_segments": []}, ran)

+    # SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
+    # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded by
+    # _cloud_available so we fail clearly rather than silently running local); None / unknown ?
default.
+    choice = (req.compute or "").strip().lower()
+    if choice and choice not in ("local", "cloud"):
+        logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using
```

```
server default.",
+                       req.compute)
+        choice = ""
+    if choice == "local":
+        return None  # explicit "run on my machine" ? in-process regardless of the server's
target
+    if choice == "cloud":
+        if not _cloud_available():
+            return _failed("cloud-serving was requested but this server isn't configured for it
"
+                           "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
+                           False)
+        compute = get_compute_backend(global_config, target_override="cloud_run")
+    else:
+        compute = get_compute_backend(global_config)  # server default (default-OFF unless
cloud_run)
+    if compute is None:
+        return None  # default-OFF ? run the in-process segmenter below (unchanged)
+
+    storage_cfg = getattr(global_config, "storage", None)
+
     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
     try:
         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index ed86c87..ec484e4 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -91,10 +91,11 @@ def cloud_env(tmp_path, monkeypatch):

 def _inject_fake(monkeypatch, fake):
     """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote)
build a
-    real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token."""
+    real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token.
``**kw``
+    forwards the per-request ``target_override`` (item 3) so the cloud-picker path resolves
too."""
     real = compute_pkg.get_compute_backend
     monkeypatch.setattr(compute_pkg, "get_compute_backend",
-                        lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))
+                        lambda config, **kw: real(config, run_client=fake, policy=_FAST,
token="tok", **kw))


 def _meta(row):
@@ -227,6 +228,114 @@ def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env,
caplog):
     assert any("DIVERGED" in r.getMessage() and "!= local" in r.getMessage() for r in
caplog.records)


+class _InProcSeg:
+    """Minimal in-process segmenter (writes one play row) ? proves the local path actually
ran."""
+    def __init__(self, db_, cfg_):
+        self.db = db_
+
+    def process_video(self, vid, path, *a, **k):
+        sid = self.db.add_play_segment(vid, 0.0, 5.0)
+        return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}]
```

```
+
+
+# --- item 3: SPA compute-picker (per-request compute override + cloud_available capability)
---
+
+def test_per_request_local_forces_in_process(cloud_env):
+    """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
+    forces the in-process segmenter and NEVER dispatches to Cloud Run."""
+    db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
+
+    out = m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion", compute="local"))
+    assert out["status"] == "success"
+    assert len(out["play_segments"]) == 1
+    assert "cloud_run" not in out["message"]
+    assert fake.calls == []  # explicit "run on my machine" ? no dispatch despite a cloud
server
+
+
+def test_per_request_cloud_forces_dispatch_on_local_server(tmp_path, monkeypatch):
+    """Server default is LOCAL (compute_target=''), but a per-request compute='cloud' forces a
Cloud
+    Run dispatch when the control plane is wired (CLOUD_RUN_JOB + GOOGLE_CLOUD_PROJECT + a
bucket)."""
+    db = Database(str(tmp_path / "t.db"))
+    bucket = tmp_path / "bucket"
+    bucket.mkdir()
+    cfg = AppConfig.model_validate({
+        "deployment": {"compute_target": ""},  # server default = in-process
+        "storage": {"backend": "local", "output_root": str(bucket)},
+    })
+    monkeypatch.setattr(m, "db_instance", db)
+    monkeypatch.setattr(m, "global_config", cfg)
+    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
+    fake_local = tmp_path / "fetched.mp4"
+    fake_local.write_bytes(b"FAKEVIDEOBYTES")
+    monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg: str(fake_local))
+    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
+    monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+
+    out = m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="cloud"))
+    assert out["status"] == "success"
+    assert len(out["play_segments"]) == 2
+    assert len(fake.calls) == 1  # per-request override dispatched despite the LOCAL server
default
+
+
+def test_per_request_cloud_unconfigured_fails_clearly(tmp_path, monkeypatch):
+    """compute='cloud' on a server NOT wired for it fails clearly (no silent local fallback, no
+    dispatch) so the picker can surface 'cloud isn't available here'."""
+    db = Database(str(tmp_path / "t.db"))
+    cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}})  # no bucket, no env
+    monkeypatch.setattr(m, "db_instance", db)
+    monkeypatch.setattr(m, "global_config", cfg)
+    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
```

```
+    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
+    video = tmp_path / "v.mp4"
+    video.write_bytes(b"hashable-bytes")
+
+    out = m._execute_processing(
+        m.ProcessRequest(video_path=str(video), segmenter="wasb_hybrid", compute="cloud"))
+    assert out["status"] == "failed"
+    assert "isn't configured" in out["message"]
+
+
+def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
+    """An unrecognised compute value is ignored (logged) and the server default applies ? here
a
+    cloud_run server still dispatches to the cloud."""
+    db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    with caplog.at_level("WARNING", logger="backend.main"):
+        out = m._execute_processing(
+            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="banana"))
+    assert out["status"] == "success"
+    assert len(fake.calls) == 1  # fell back to the configured cloud_run default
+    assert any("unknown compute" in r.getMessage() for r in caplog.records)
+
+
+def test_capabilities_reports_cloud_available(cloud_env):
+    """A cloud_run-configured server advertises deployment.cloud_available=True (the SPA shows
the
+    cloud toggle only then)."""
+    _db, _bucket, _monkeypatch = cloud_env
+    caps = m.capabilities()
+    assert caps["deployment"]["cloud_available"] is True
+    assert caps["deployment"]["compute_target"] == "cloud_run"
+
+
+def test_capabilities_cloud_unavailable_by_default(monkeypatch):
+    """A default (local) server with no Cloud Run env advertises cloud_available=False."""
+    monkeypatch.setattr(m, "global_config", AppConfig())
+    monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
+    monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
+    caps = m.capabilities()
+    assert caps["deployment"]["cloud_available"] is False
+
+
 def test_full_api_process_to_query_via_cloud(cloud_env):
     """End-to-end through the public API: POST /api/process (gs:// input) ? the inline worker
dispatches
     to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
```

4. assistant (2026-06-22T15:43:47.442Z)

Let me read the surrounding code in `backend/main.py` to understand the full context, particularly the call site and the watermark/cleanup logic.

5. user (2026-06-22T15:43:48.447Z)

```
486         return n
487
488
489     def _cloud_available() -> bool:
490         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
491
492         True when the control plane is wired for Cloud Run: either the configured default
```

```
target is
493         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
494         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
495         are present so a forced per-request override can actually dispatch + collect a
result.
496
497         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
498         deployment = getattr(global_config, "deployment", None)
499         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
500             return True
501         has_job = bool(os.environ.get("CLOUD_RUN_JOB") and
os.environ.get("GOOGLE_CLOUD_PROJECT"))
502         out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
503         return bool(has_job and out_root)
504
505
506     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
507                                play_wm: int, cand_wm: int, notify) ->
Optional[Tuple[Dict[str, Any], bool]]:
508         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
509         == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
510         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
511
512         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
513         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
514         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
515         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
516         the in-process path stays byte-identical)."""
517         from backend.compute import ComputeJobSpec, get_compute_backend
518
519         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
520             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
521             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
522             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
523             return ({"video_id": video_id, "status": "failed", "message": msg,
524                      "results": [r.model_dump() for r in val_results], "play_segments": []},
ran)
525
526         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
527         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
528         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
529         choice = (req.compute or "").strip().lower()
530         if choice and choice not in ("local", "cloud"):
531             logger.warning("[API] ignoring unknown compute=%r (expected 'local'|'cloud');
using server default.",
532                            req.compute)
533             choice = ""
534         if choice == "local":
535             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
```

```
        if choice == "cloud":
            if not _cloud_available():
                return _failed("cloud-serving was requested but this server isn't configured
for it "
                               "(no Cloud Run target). Pick 'run on my machine', or
configure cloud serving.",
                               False)
            compute = get_compute_backend(global_config, target_override="cloud_run")
        else:
            compute = get_compute_backend(global_config)  # server default (default-OFF
unless cloud_run)
        if compute is None:
            return None  # default-OFF ? run the in-process segmenter below (unchanged)

        storage_cfg = getattr(global_config, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}",
False)
        if input_ref.backend == "local":
            return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
                           "can't be read by the Cloud Run job. Put the video in your bucket
first.", False)
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        if not out_root:
            return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
                           "? that's where the Cloud Run job writes the result.", False)

        notify("dispatching (cloud_run)")
        logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
        spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
        try:
            result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
            return _failed(f"cloud dispatch error: {e}", True)
        if result.returncode != 0:
            return _failed(f"cloud job failed (returncode {result.returncode}).", True)
        if result.video_id and result.video_id != video_id:
            # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
            # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
            logger.warning("[API] cloud video_id %s != local %s ? input bytes differ between
the API hash "
                           "and the worker hash; ingesting under the local id.",
result.video_id, video_id)

        notify("ingesting (cloud_run)")
        storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
        try:
            _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
        except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
```

```
583             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
584             return _failed(f"cloud-serving: could not read the result from
{result.outputs_uri}: {e}", True)
585
586         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
587         segments = [s for s in db_instance.query_play_segments(video_id, {}) if
int(s.get("id", 0)) > play_wm]
588         _finalize_reingest(video_id, segments, play_wm, cand_wm)
589         return ({"video_id": video_id, "status": "success",
590                  "message": f"Successfully indexed {len(segments)} play segments (cloud_run
/ '{seg_name}').",
591                  "results": [r.model_dump() for r in val_results], "play_segments":
segments}, True)
592
593
594     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
595         """The full hash ? validate ? segment ? report pipeline for one video.
596
597         This is the former synchronous /api/process body, unchanged in behavior, now run on
598         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
599         the sync endpoint used to return (UI compatibility); the per-video observability
600         report is still emitted in `finally:` and grafted as response["reports"].
601
602         `progress` is an optional callable(stage: str) used to surface coarse job progress.
603         Raises on unexpected internal errors ? the caller records those as a job failure
604         (after this function has already restored the pre-run segments, see below).
605         """
606         notify = progress or (lambda stage: None)
607
608         notify("hashing")
609         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
610         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
611         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
612         # consumers (hash / validators / segmenter) use the resolved local path.
613         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
614         video_id = compute_video_id(local_video)
615         was_reingest = db_instance.get_video(video_id) is not None
616
617         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
618         notify("validating")
619         logger.info(f"Running validations for {req.video_path}...")
620         val_results, val_context = registry.run_all_with_context(local_video, global_config)
621         failures = [r for r in val_results if not r.passed]
622
623         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
624         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
625         seg_name = req.segmenter or default_seg
626         segmenter_actual = None
627         segmentation_ran = False
628         response: Optional[Dict[str, Any]] = None
629         try:
630             if failures:
631                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
632                 # status; it no longer destroys the video's prior good segments.
633                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
634                 response = {
635                     "video_id": video_id,
```

```
636                     "status": "failed",
637                     "message": "Video failed validation checks.",
638                     "results": [r.model_dump() for r in val_results],
639                 }
640                 return response
641 
642             # 2. Run segmenter & indexer
643             logger.info("[API] Video passed checks. Segmenting and indexing...")
644             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
645 
646             segmenter_cls = segmenter_registry.get(seg_name)
647             if not segmenter_cls:
648                 # Submit-time validation already 400s unknown names; this is the defensive
649                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
650                 available = list(segmenter_registry.list_available().keys())
651                 response = {
652                     "video_id": video_id,
653                     "status": "failed",
654                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
655                     "results": [r.model_dump() for r in val_results],
656                     "play_segments": [],
657                 }
658                 return response
659 
660             logger.info(f"[API] Using segmenter: {seg_name}")
661             notify(f"segmenting ({seg_name})")
662             segmenter_actual = seg_name
663             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
664             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
665             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
666             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
667             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
668             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
669             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
670             # they're dropped on the cloud path in this slice ? a documented follow-up.)
671             remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
672             if remote is not None:
673                 response, segmentation_ran = remote  # `ran` = did the cloud job actually
execute (report)
674                 return response
675             segmenter = segmenter_cls(db_instance, global_config)
676             try:
677                 result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
678             except Exception:
679                 # A raising segmenter must not leave half-written rows: restore the pre-run
680                 # state (same destroy-after-confirm contract as the Failure branch), then
let
681                 # the job worker record the failure.
682                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
683                 raise
684             segmentation_ran = True
685 
686             if isinstance(result, Failure):
687                 logger.error(f"[API] Segmenter failed: {result.message}")
```

```
688                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
689                 response = {
690                     "video_id": video_id,
691                     "status": "failed",
692                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
693                     "results": [r.model_dump() for r in val_results],
694                     "play_segments": [],
695                 }
696                 return response
697 
698             segments = result.value if hasattr(result, "value") else result
699             notify("finalizing")
700             _finalize_reingest(video_id, segments, play_wm, cand_wm)
701 
702             response = {
703                 "video_id": video_id,
704                 "status": "success",
705                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
706                 "results": [r.model_dump() for r in val_results],
707                 "play_segments": segments,
708             }
709             return response
710         finally:
711             # Always emit the per-video observability report (next to the video, or to
712             # report.output_dir). Never raises. Surface the paths in the response.
713             paths = emit_indexer_report(
714                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
715                 val_results=val_results, val_context=val_context,
```

6. assistant (2026-06-22T15:43:51.512Z)

Now let me look at the `prune_segments`, `segment_watermarks`, and `_finalize_reingest` to understand the destroy-after-confirm and whether `_failed()` is safe when no rows were written.

7. user (2026-06-22T15:43:52.517Z)

```
backend\infrastructure\database.py:424:    def segment_watermarks(self, video_id: str) ->
Tuple[int, int]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-425-
"""Return (max play_segment id, max candidate_segment id) for a video (0 if none).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-426-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-427-
Captured before a (re)segmentation so the run's new rows (id > watermark) can be
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-428-
told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-429-
"""
backend\infrastructure\database.py-430-        with self._connect() as conn:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-431-
cur = conn.cursor()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-432-
p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id = ?",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-433-
(video_id,)).fetchone()[0]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-434-
c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE video_id = ?",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-435-
(video_id,)).fetchone()[0]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-436-
return int(p), int(c)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-437-
backend\infrastructure\database.py:438:    def prune_segments(self, video_id: str, *, play_upto:
Optional[int] = None,
```

```
backend\infrastructure\database.py-439-                        play_after: Optional[int] = None,
cand_upto: Optional[int] = None,
backend\infrastructure\database.py-440-                        cand_after: Optional[int] = None)
-> None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-441-
"""Delete play/candidate segments by id range (events cascade from play_segments).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-442-
backend\infrastructure\database.py-443-        Used to finish a (re)segmentation: keep the NEW
rows and drop the OLD ones on a
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-444-
successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial rows
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-445-
and keep the OLD ones when the run failed/produced nothing (``*_after``).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-446-
"""
backend\infrastructure\database.py-447-        with self._connect() as conn:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-448-
cur = conn.cursor()
backend\infrastructure\database.py-449-            if play_upto is not None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-450-
cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?", (video_id, play_upto))
backend\infrastructure\database.py-451-            if play_after is not None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-452-
cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?", (video_id, play_after))
backend\infrastructure\database.py-453-            if cand_upto is not None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-454-
cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?", (video_id,
cand_upto))
backend\infrastructure\database.py-455-            if cand_after is not None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-456-
cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id > ?", (video_id,
cand_after))
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-457-
conn.commit()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-458-
backend\infrastructure\database.py-459-    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py-460-        try:
backend\infrastructure\database.py-461-            with self._connect() as conn:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-462-
cursor = conn.cursor()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py-463-
cursor.execute(
--
backend\main.py:283:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int)
-> None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-284-    """Commit a
(re)segmentation with destroy-AFTER-confirm semantics.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-285-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-286-    On a non-empty run,
drop the PRIOR rows (id <= the pre-run watermark) so only the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-287-    fresh segments
remain. On an empty run, drop the NEW partial rows (id > watermark) and
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-288-    keep the prior good
results intact ? a failed/empty re-run never destroys prior data.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-289-    """
backend\main.py-290-    if segments:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-291-
db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
backend\main.py-292-    else:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-293-
db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-294-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-295-
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-296-# --- API Endpoints ---
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-297-@app.get("/api/config")
backend\main.py-298-def get_config():
backend\main.py-299-    # P0b: this endpoint is unauthenticated, so never serialize a
credential-shaped field. Today
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-300-    # AppConfig holds no
secrets (the Gemini key is env-only; cf_access_aud is a public id), so this
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-301-    # is byte-identical
for the real fields ? but it fails-closed if a secret is ever added (guarded
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-302-    # by
tests/test_redact.py + the /api/config redaction test).
backend\main.py-303-    if global_config is None:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-304-        return {}
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-305-    return
redact_secrets(global_config.model_dump(mode="json"))
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-306-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-307-@app.get("/api/health")
backend\main.py-308-def health():
```

8. assistant (2026-06-22T15:43:56.401Z)

I notice the grep returned results from a sibling repo path (`badminton-highlight-indexer`) intermixed ? but the worktree is `bhi-picker`. Let me confirm the relevant code is in the bhi-picker worktree and read the `capabilities()` function and `ProcessRequest` there directly.

9. user (2026-06-22T15:43:57.113Z)

```
355         ffprobe = shutil.which(ffprobe_bin())
356         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
357         wasb = getattr(indexing, "wasb", None)
358         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
359         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
360         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
361         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
362         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
363         nvidia_smi = shutil.which("nvidia-smi")
364         return {
365             "api_version": API_VERSION,
366             "engine_version": _engine_version(),
367             "segmenters": sorted(segmenter_registry.list_available().keys()),
368             "default_segmenter": getattr(indexing, "default_segmenter", None),
369             "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
370                        "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
371             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
372             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
373             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
374             "deployment": {
375                 "profile": getattr(deployment, "profile", "") or "",
376                 "compute_target": getattr(deployment, "compute_target", "") or "",
377                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
378                 # the "run in the cloud" toggle only when true (else it's local-only).
379                 "cloud_available": _cloud_available(),
380             },
381         }
382
383
384     @app.get("/api/videos")
```

10. assistant (2026-06-22T15:44:01.046Z)

Good ? the code is in bhi-picker. The grep earlier picked up the sibling repo because it has additional working dirs. Let me read the `_finalize_reingest`, `ProcessRequest` and submit-time validation in the bhi-picker worktree, plus where `/api/process` validates the request (to check whether `compute` is validated at submit time).

11. user (2026-06-22T15:44:01.556Z)

```
255     # Legacy thin wrapper for any remaining direct calls during transition.
256     # New code should import load_app_config / AppConfig from backend.config directly.
257     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
258         cfg = load_app_config(config_path)
259         return cfg.model_dump(mode="json")
260
261     # --- API Models ---
262     class ProcessRequest(BaseModel):
263         video_path: str
264         player_pool: Optional[List[str]] = None
265         max_frames: Optional[int] = None
266         segmenter: Optional[str] = None
267         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
268         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
269         locale: Optional[str] = None
270         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
271         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
272         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
273         compute: Optional[str] = None
274
275     class QueryRequest(BaseModel):
276         video_id: str
277         server: Optional[str] = None
278         receiver: Optional[str] = None
279         duration_min: Optional[float] = None
280         event_type: Optional[str] = None
281
282     class CompileRequest(BaseModel):
283         video_id: str
284         segment_ids: List[int]
285         output_filename: str
286         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
287         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
288         locale: Optional[str] = None
289
290     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
291         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
292
293         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
294         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
```

12. assistant (2026-06-22T15:44:05.916Z)

Now let me look at how `/api/process` accepts the request and whether there's any submit-time validation of the `compute` field, and where `db_instance.add_video` is called relative to `_maybe_dispatch_remote`. Let me also check the `get_compute_backend` full function and the `DEFAULT_POLICY`/`token` handling for the `""` empty-string concern.

13. user (2026-06-22T15:44:06.724Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38          target_override: Optional[str] = None,
39      ) -> Optional[ComputeBackend]:
40          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
41          in-process (the default-OFF path).
42
43          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
44          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
45          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
46          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
47
48          ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
49          per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
50          server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
51          """
```

```
52          target = (
53              target_override
54              if target_override is not None
55              else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
56          )
57          if target != "cloud_run":
58              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
59
60          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
61          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
62
63          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
64          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
65          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
66          client = run_client if run_client is not None else GoogleCloudRunJobClient()
67          return CloudRunCompute(
68              run_client=client, job_name=job_name, region=region, project=project,
69              storage_config=_storage_dict(config), policy=policy, token=token,
70          )
71
```

14. assistant (2026-06-22T15:44:09.795Z)

Now let me check the `/api/process` endpoint to see how `ProcessRequest` validation happens at submit time (whether `compute` is validated, and whether `add_video` already wrote a row before `_maybe_dispatch_remote`).

15. user (2026-06-22T15:44:11.382Z)

```
262:class ProcessRequest(BaseModel):
506:def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
594:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
597:    This is the former synchronous /api/process body, unchanged in behavior, now run on
723:@app.post("/api/process", status_code=202)
724:def process_video(req: ProcessRequest, request: Request):
924:        # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
LOCAL file
```

16. user (2026-06-22T15:44:13.964Z)

```
723     @app.post("/api/process", status_code=202)
724     def process_video(req: ProcessRequest, request: Request):
725         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
726
727         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
728         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
729         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
730         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
731         """
732         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
733         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
734         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
735         try:
736             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
737         except edge_auth.EdgeAuthError as e:
738             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
739
740         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
741         try:
```

```
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames, owner_id=owner_id,
                                      locale=req.locale):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}/cost")
    def get_job_cost(job_id: str):
        """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
        GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
        null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
        cost = db_instance.get_job_cost(job_id)
        if cost is None:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        fx = global_config.billing.fx_inr_per_usd
        usd = cost.get("cost_usd")
        cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
        cost["fx_inr_per_usd"] = fx
        return cost


    @app.get("/api/videos/{video_id}/cost")
    def get_video_cost(video_id: str):
        """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
        per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
```

```
794         agg = db_instance.get_video_cost(video_id)
795         fx = global_config.billing.fx_inr_per_usd
796         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
797         agg["fx_inr_per_usd"] = fx
798         return agg
799
800
801     @app.get("/api/jobs/{job_id}")
802     def get_job(job_id: str):
```

17. assistant (2026-06-22T15:44:17.479Z)

Now I notice an important detail: at submit time, `create_job` does NOT persist `req.compute`. Let me verify the `create_job` signature and how `_execute_processing` reconstructs the `ProcessRequest` from the job row ? because if `compute` is dropped between submit and the worker, the picker would never take effect.

18. user (2026-06-22T15:44:18.943Z)

```
238:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
246:    _drain_due_jobs,
262:class ProcessRequest(BaseModel):
594:def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
730:    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
764:    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
768:    _get_executor().submit(_drain_due_jobs)
1270:         _get_executor().submit(_drain_due_jobs)
1278:    # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
```

19. assistant (2026-06-22T15:44:22.991Z)

The job-draining and `ProcessRequest` reconstruction happen in the `job_worker` module (imported at line 246). Let me find where `_execute_processing` is invoked with a reconstructed `ProcessRequest`, to check whether `compute` and `locale` survive the submit?worker boundary.

20. user (2026-06-22T15:44:24.104Z)

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\bhi-picker\backend\job_worker.py.
Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-
indexer.</tool_use_error>
```

21. user (2026-06-22T15:44:28.804Z)

```
238     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
239     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
240     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
241     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
242     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
243     from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
244         JobWaiting,
245         _arm_wake_timer,
246         _drain_due_jobs,
247         _get_executor,
248         _job_to_request,
249         _MAX_DRAIN_WAKE_SEC,
250         _maybe_record_job_cost,
251         _run_claimed_job,
```

22. assistant (2026-06-22T15:44:32.472Z)

There's a `_job_to_request` seam. This is the critical path: if `compute` isn't persisted in

`create_job` and reconstructed in `_job_to_request`, the picker silently never works via the real `/api/process` endpoint (only via direct `_execute_processing` calls as the tests do). Let me examine both.

23. user (2026-06-22T15:44:33.909Z)

```
56-        self.progress = progress
57-
58-
59:def _job_to_request(job: Dict[str, Any]):
60-    """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61-    re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62-    the original submit. The row stores exactly the ProcessRequest fields."""
63-    import backend.main as _main
64:    return _main.ProcessRequest(
65-        video_path=job["video_path"],
66-        player_pool=job.get("player_pool"),
67-        max_frames=job.get("max_frames"),
--
100-
101-def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
102-    """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
103:    ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
104-    (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
105-    persist it.
106-
--
121-        usage = (result or {}).get("usage") or {}
122-        _main.db_instance.record_job_cost(
123-            job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
124:            gpu_type=usage.get("gpu_type"), compute_backend=(result or
{}).get("compute_backend"),
125-            owner_id=job.get("owner_id"), margin=billing_cfg.margin,
126-        )
127-        logger.info("Recorded cost for job %s (pricebook=%s).", job["id"],
billing_cfg.pricebook_version)
--
161-    try:
162-        delay = _main.db_instance.seconds_until_next_due()
163-    except Exception as e:  # pragma: no cover - defensive; never wedge the worker
164:        logger.error(f"Could not compute next-due delay: {e}")
165-        return
166-    if delay is None:
167-        return  # nothing waiting
```

24. assistant (2026-06-22T15:44:38.106Z)

This is the key finding. `_job_to_request` reconstructs the `ProcessRequest` from the persisted job row but does NOT set `compute=` (or `locale=`). Let me read the full `_job_to_request` to confirm the exact fields, then check `create_job` to confirm `compute` is not even persisted.

25. user (2026-06-22T15:44:38.892Z)

```
59      def _job_to_request(job: Dict[str, Any]):
60          """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61          re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62          the original submit. The row stores exactly the ProcessRequest fields."""
63          import backend.main as _main
64          return _main.ProcessRequest(
65              video_path=job["video_path"],
66              player_pool=job.get("player_pool"),
67              max_frames=job.get("max_frames"),
68              segmenter=job.get("segmenter"),
```

```
69          )
70
71
72      def _run_claimed_job(job: Dict[str, Any]) -> None:
73          """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
```

26. assistant (2026-06-22T15:44:42.387Z)

Confirmed. `_job_to_request` reconstructs `ProcessRequest` with only `video_path`, `player_pool`, `max_frames`, `segmenter`. It does NOT pass `compute` (nor `locale`, but `locale` is a pre-existing separate concern). Now let me verify `create_job`'s signature to confirm `compute` is never persisted ? meaning even if `_job_to_request` were updated, the column wouldn't exist.

27. user (2026-06-22T15:44:43.861Z)

```
623:    def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
624-                   player_pool: Optional[List[str]] = None,
625-                   max_frames: Optional[int] = None,
626-                   policy: Optional[Dict[str, Any]] = None,
627-                   owner_id: Optional[str] = None,
628-                   locale: Optional[str] = None) -> bool:
629-        """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
630-        (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
631-        (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
632-        ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded."""
633-        return self._execute_write(
634-            "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames, status,
policy, "
635-            "owner_id, locale) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?, ?)",
636-            (job_id, video_path, segmenter,
637-             json.dumps(player_pool) if player_pool is not None else None,
638-             max_frames, json.dumps(policy) if policy is not None else None, owner_id,
locale),
639-            f"Failed to create job {job_id}",
640-        )
641-
642-    def count_jobs_by_locale(self) -> Dict[str, int]:
643-        """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
644-        CLI \ pre-v7 \ older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
645-        with self._connect() as conn:
646-            rows = conn.cursor().execute(
647-                "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
648-                "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
649-            ).fetchall()
650-        return {str(r[0]): int(r[1]) for r in rows}
651-
652-    def mark_job_running(self, job_id: str) -> bool:
653-        return self._execute_write(
654-            "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
655-            "progress='starting' WHERE id = ?", (job_id,),
656-            f"Failed to mark job {job_id} running",
657-        )
658-
```

28. assistant (2026-06-22T15:44:50.898Z)

Confirmed. The `jobs` table has no `compute` column, `create_job` doesn't accept or persist it, and `_job_to_request` doesn't restore it. This is the central bug. Let me verify the full flow path from `/api/process` ? worker is the ONLY production path (i.e., there's no synchronous path that calls `_execute_processing` directly with the original `req`). The endpoint returns 202 and submits to a worker, so yes ? the worker reconstruction is the only path.

Let me also double-check whether the tests catch this. The new tests call `m._execute_processing(m.ProcessRequest(..., compute="local"))` directly ? bypassing the submit?persist?reconstruct boundary. So they pass while the real endpoint silently drops `compute`. Let me confirm there's no end-to-end test through `/api/process` exercising `compute`.

29. user (2026-06-22T15:44:52.400Z)

```
3:Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the
rally
16:from fastapi.testclient import TestClient
244:    """Server configured for cloud (compute_target=cloud_run), but a per-request
compute='local'
252:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="motion", compute="local"))
260:    """Server default is LOCAL (compute_target=''), but a per-request compute='cloud' forces
a Cloud
282:        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="cloud"))
289:    """compute='cloud' on a server NOT wired for it fails clearly (no silent local fallback,
no
302:        m.ProcessRequest(video_path=str(video), segmenter="wasb_hybrid", compute="cloud"))
315:            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid",
compute="banana"))
340:    """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
345:    client = TestClient(m.app)
347:    r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
356:    q = client.post("/api/query", json={"video_id": video_id})
```

30. assistant (2026-06-22T15:44:58.552Z)

Confirmed. The one true end-to-end test through `/api/process` (line 347) does NOT pass `compute`, and all the new `compute=` tests call `_execute_processing` directly, bypassing the persist/reconstruct boundary. So the picker is dead on the real endpoint and no test catches it.

Now let me verify the remaining probe areas. Let me check the `_failed()` call safety in the unconfigured-cloud branch (it runs `prune_segments` with `play_after`/`cand_after`), and check whether `add_video` already wrote a "processed" row before `_maybe_dispatch_remote`. Looking back at `_execute_processing`: line 644 calls `add_video(..., "processed", ...)` BEFORE line 671's `_maybe_dispatch_remote`. So on the unconfigured-cloud `_failed("isn't configured")` path, the video row is left as "processed" but the response says "failed". Let me check what the in-process failure branches do with the video status for comparison.

31. user (2026-06-22T15:44:59.604Z)

```
715                 val_results=val_results, val_context=val_context,
716                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
717                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
718             )
719             if paths and isinstance(response, dict):
720                 response["reports"] = paths
721
722
723     @app.post("/api/process", status_code=202)
724     def process_video(req: ProcessRequest, request: Request):
```

32. assistant (2026-06-22T15:45:09.662Z)

Now I understand the video-status behavior. The existing in-process segmenter-failure branches (lines 686-696) also leave the video row as "processed" (set at line 644) while returning a "failed" response ? they only prune segments, never flip the video status back. So the unconfigured-cloud `_failed()` path is consistent with the existing failure branches: same "processed" row + pruned segments + "failed" response. That's a pre-existing behavior, not a regression introduced by this change. The `prune_segments` with no new rows written is a harmless no-op (DELETE matching zero rows). So that probe area is clean.

Let me verify the default-OFF invariant precisely by confirming that `compute=None` produces `choice=""`, which routes to the `else` branch calling `get_compute_backend(global_config)` ? byte-identical to the old code. Let me re-examine lines 529-545 logic once more and the old code. The old code was:
```python
compute = get_compute_backend(global_config)
if compute is None: return None
storage_cfg = ...
```
New code with `compute=None`: `choice=""`, skips both `if choice==...`, hits `else: compute = get_compute_backend(global_config)`, then `if compute is None: return None`. Identical. Good ? default-OFF is preserved within `_maybe_dispatch_remote`.

Now let me check the `target_override=""` empty-string concern. Could any caller pass `""`? In `_maybe_dispatch_remote`, the only call with an override is `get_compute_backend(global_config, target_override="cloud_run")` (literal). The `else` branch passes no override (`None`). So no `""` is passed. The empty-string-as-explicit-override is a latent footgun but has no live caller. Let me confirm there are no other callers of `get_compute_backend` with a dynamic `target_override`.

33. user (2026-06-22T15:45:11.058Z)

C:\Users\avidu\Projects\bhi-picker\backend\worker__main__.py:168:   compute = get_compute_backend(config)
C:\Users\avidu\Projects\bhi-picker\backend\main.py:541:   compute = get_compute_backend(global_config, target_override="cloud_run")
C:\Users\avidu\Projects\bhi-picker\backend\main.py:543:   compute = get_compute_backend(global_config) # server default (default-OFF unless cloud_run)
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:32:def get_compute_backend(
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:38:   target_override: Optional[str] = None,
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:48:   ``target_override`` lets a single request force a target (the SPA compute-picker maps a
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:49:   per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating the
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:53:   target_override
C:\Users\avidu\Projects\bhi-picker\backend\compute__init__.py:54:   if target_override is not None

34. assistant (2026-06-22T15:45:17.673Z)

Confirmed: only one call passes `target_override`, and it's the literal `"cloud_run"`. The `""` footgun has no live caller. That probe area is clean (no real bug).

Now let me verify the capabilities/security probe ? `_cloud_available()` and `capabilities()` only return booleans and the already-exposed `compute_target` string, no secrets. `_cloud_available()` reads env var *presence* via `bool(...)` and `output_root` presence via `.strip()` truthiness but returns only a bool. The `capabilities()` endpoint already exposed `compute_target` before this change. No secret leak. Clean.

Let me confirm `capabilities()` is reachable unauthenticated and whether `cloud_available` could leak topology ? but it's just a boolean, same risk class as the pre-existing `compute_target`. Clean.

Now let me run the test file to confirm the new tests pass (proving they don't exercise the

broken persist path) and the suite is green.

35. user (2026-06-22T15:45:23.197Z)

```
..............                                                           [100%]
14 passed in 3.07s
```

36. assistant (2026-06-22T15:45:27.164Z)

All 14 pass ? because they bypass the submit?persist?reconstruct boundary. Let me write a quick reproduction to confirm the bug: submit through `/api/process` with `compute` and show it's dropped by `_job_to_request`. Let me check what `claim_due_job` returns (does the job dict even have a `compute` key?) and confirm the reconstruction drops it.

37. user (2026-06-22T15:45:28.742Z)

```
def _job_to_request(job: Dict[str, Any]):
    """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
    re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
    the original submit. The row stores exactly the ProcessRequest fields."""
    import backend.main as _main
    return _main.ProcessRequest(
        video_path=job["video_path"],
        player_pool=job.get("player_pool"),
        max_frames=job.get("max_frames"),
        segmenter=job.get("segmenter"),
    )

--- create_job persists compute? ---
False
```

38. assistant (2026-06-22T15:45:32.692Z)

Confirmed conclusively. Let me write an end-to-end reproduction proving that a `compute="local"` submitted through `/api/process` is silently ignored by the worker (the picker is non-functional on the real endpoint). Let me check the test fixtures to build a minimal e2e repro using the existing `cloud_env`.

39. user (2026-06-22T15:45:33.371Z)

```
338
339     def test_full_api_process_to_query_via_cloud(cloud_env):
340         """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
341         to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
342         db, _bucket, monkeypatch = cloud_env
343         fake = _FakeCloudClient(video_id="vidCloud2")
344         _inject_fake(monkeypatch, fake)
345         client = TestClient(m.app)
346
347         r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
348         assert r.status_code == 202
349         job_id = r.json()["job_id"]
350
351         job = client.get(f"/api/jobs/{job_id}").json()
352         assert job["status"] == "done"
353         video_id = job["video_id"]
354         assert video_id
355
356         q = client.post("/api/query", json={"video_id": video_id})
357         assert q.status_code == 200
358         segs = q.json()["play_segments"]
359         assert len(segs) == 2
```

```
360         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
361
```

40. assistant (2026-06-22T15:45:38.303Z)

This e2e test confirms the model: `cloud_env` configures `compute_target=cloud_run` so the *server default* dispatches to cloud. The picker would only matter when a request overrides the default. Let me write a definitive repro: a cloud-configured server + `/api/process` with `compute="local"` ? if the picker worked, it would run in-process and NOT dispatch; with the bug, it dispatches to cloud (the override is lost).

41. user (2026-06-22T15:45:50.370Z)

```
        r = client.post("/api/process",
                        json={"video_path": "gs://b/clip.mp4", "segmenter": "motion", "compute":
"local"})
        assert r.status_code == 202
        job_id = r.json()["job_id"]
        job = client.get(f"/api/jobs/{job_id}").json()
        print("job status:", job["status"])
        print("fake cloud dispatch calls:", len(fake.calls))
        # The user asked for LOCAL. If the picker worked, fake.calls would be 0.
>       assert len(fake.calls) == 0, "BUG: compute='local' override was DROPPED; dispatched to
cloud anyway"
E       AssertionError: BUG: compute='local' override was DROPPED; dispatched to cloud anyway
E       assert 1 == 0
E        +  where 1 = len([['infer', '--video-uri', 'gs://b/clip.mp4', '--out-uri',
'C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-891\\test_repro_picker_dropped_e2e0\\bucket', '--done-uri', ...]])
E        +    where [['infer', '--video-uri', 'gs://b/clip.mp4', '--out-uri',
'C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-891\\test_repro_picker_dropped_e2e0\\bucket', '--done-uri', ...]] =
<tests.test_api_cloud_dispatch._FakeCloudClient object at 0x000001FD1D321550>.calls

..\..\AppData\Local\Temp\repro_picker.py:25: AssertionError
------------------------------ Captured log call ------------------------------
WARNING  backend.main:main.py:575 [API] cloud video_id vidX != local
b708f3d1de52d89c6d374c6228a17178 ? input bytes differ between the API hash and the worker hash;
ingesting under the local id.
=========================== short test summary info ===========================
FAILED ..\..\AppData\Local\Temp\repro_picker.py::test_repro_picker_dropped_e2e
1 failed in 4.32s
```

42. assistant (2026-06-22T15:45:54.520Z)

Reproduced. A user submitting `compute="local"` through the real `/api/process` endpoint on a cloud-configured server gets dispatched to Cloud Run anyway ? the override is silently dropped because `create_job` never persists `compute` and `_job_to_request` never restores it. The whole picker feature is non-functional through the public API.

Let me clean up the repro file and do a final scan of the remaining probe areas (ordering of failure messages) to be thorough.

43. user (2026-06-22T15:45:55.039Z)

cleaned

44. assistant (2026-06-22T15:46:27.631Z)

On the **ordering** probe: in `_maybe_dispatch_remote`, the picker logic (lines 526-545) runs before input-resolution/bucket checks (lines 549-560). For `compute="cloud"` on an unconfigured server, `_cloud_available()` returns False and fails with "isn't configured" before reaching input-type errors. That precedence is sensible: a config-not-available error is more actionable than "your input must be gs://". And `_cloud_available()` requiring `output_root` means the

later "needs output_root" check at line 558 is effectively pre-checked. No bug, though there's a minor redundancy (input being non-cloud is still checked later, correctly). Clean.

Now I have a complete picture. Here are my findings.

Adversarial review findings ? per-request compute picker

BLOCKER ? the `compute` override is silently dropped between submit and worker; the feature is dead on the real `/api/process` endp

Where: the field is added at `backend/main.py:273` and consumed at `backend/main.py:529`, but the submit?worker boundary never carries it:
- `backend/api/job_worker.py:64-69` ? `_job_to_request()` reconstructs the `ProcessRequest` with only `video_path`, `player_pool`, `max_frames`, `segmenter`. **`compute` is not passed**, so it defaults back to `None`.
- `backend/infrastructure/database.py:623-640` ? `create_job()` has no `compute` parameter and the `jobs` INSERT has no `compute` column, so even the submit endpoint (`backend/main.py:764`) never persists it.

Why it's a real bug: `/api/process` is an async 202 endpoint ? it persists a job row and returns; the worker later reconstructs the request from that row via `_job_to_request` and runs `_execute_processing`. Since `compute` is never persisted nor reconstructed, every request that goes through the public API loses its picker choice. `_maybe_dispatch_remote` always sees `req.compute is None` ? falls to the server default. The picker has **zero effect** in production.

Proof: I ran an end-to-end repro through `TestClient` (`POST /api/process` with `{"compute":"local"}` on a `compute_target=cloud_run` server). The user asked for local; the request dispatched to (fake) Cloud Run anyway ? `fake.calls == 1`, override lost. The same applies to `compute="cloud"` on a local-default server (it would be ignored and run in-process).

Why the test suite misses it: all 14 tests in `tests/test_api_cloud_dispatch.py` that exercise `compute=` call `m._execute_processing(m.ProcessRequest(..., compute=...))` **directly** (lines 252, 282, 302, 315), bypassing the persist/reconstruct boundary. The only true `/api/process` e2e test (`test_full_api_process_to_query_via_cloud`, line 347) doesn't pass `compute`. So the feature looks covered but its single production code path is untested.

Concrete fix: thread `compute` through the whole boundary:
1. `create_job(...)` ? add a `compute: Optional[str] = None` param; add a `compute` column to the `jobs` table (migration in `database.py`) and to the INSERT.
2. `process_video` (`backend/main.py:764`) ? pass `compute=req.compute` into `create_job`.
3. `_job_to_request` (`backend/api/job_worker.py:64`) ? add `compute=job.get("compute")`.
(Note: `locale` has the same shape but IS persisted ? `create_job` takes `locale` and the endpoint passes it ? yet `_job_to_request` also drops `locale`; that's a pre-existing, separate issue, not part of this change. Flagging only so the `compute` fix mirrors the persisted-but-restore-it pattern.)

Verdicts on the other probed risk areas ? all clean, no bug

- **Default-OFF invariant (`compute=None`):** Within `_maybe_dispatch_remote`, `compute=None` ? `choice=""` ? skips both branches ? `else: compute = get_compute_backend(global_config)` ? `if compute is None: return None`. Byte-identical to the pre-change code. Verified. (And, ironically, because of the blocker above, *every* production request is forced onto this default-OFF path ? so backward-compat is over-preserved, not broken.)

- **`compute="cloud"` on a cloud_run-default server with no env vars:** `_cloud_available()` short-circuits `True` at `main.py:499` when `compute_target == "cloud_run"`, before touching env. Correct ? no false block.

- **`compute="local"` early-return:** `return None` at `main.py:535` falls through to the in-process segmenter (`main.py:675`). No watermark/cleanup is skipped: `play_wm`/`cand_wm` are

captured at `main.py:665` *before* dispatch, and the in-process path runs its own destroy-after-confirm (`main.py:682/688/700`). The cloud path's cleanup is only relevant when the cloud path runs. Clean.

- **`_failed()` in the unconfigured-cloud branch:** `prune_segments(play_after=play_wm, cand_after=cand_wm)` at `main.py:522` with no new rows written is a no-op DELETE (matches zero rows) ? safe. The video row stays `"processed"` (set at `main.py:644`) while the response says `"failed"`, but that is **identical** to the existing in-process failure branches (`main.py:686-696`), so it's pre-existing behavior, not a regression. Coherent.

- **`target_override=""` footgun in `get_compute_backend`:** real concern in principle (`""` is treated as an explicit override ? `target != "cloud_run"` ? returns `None`), but grep confirms the only caller passing an override passes the literal `"cloud_run"` (`main.py:541`); the default path passes no override. No live caller can pass `""`. Latent, not a bug today.

- **Capability/security leak:** `_cloud_available()` returns only a `bool` (env-var *presence* via `bool(...)`, `output_root` *presence* via `.strip()` truthiness). `capabilities()` adds only that boolean; `compute_target` was already exposed pre-change. No secret values leaked.

- **Failure-message ordering:** picker/config rejection ("isn't configured") fires before input-type/bucket errors. Sensible precedence ? a config-not-available error is more actionable than an input-scheme error, and `_cloud_available()` already requires `output_root`, so the later check is consistent.

Bottom line: one blocker ? the picker is silently inoperative through `/api/process` because `compute` is never persisted/reconstructed across the async submit?worker boundary, and the tests bypass that boundary so they don't catch it. Everything else probed is sound.